TP8 – SMS Spam Classification using Bag-of-Words and TF-IDF Representations

Dr. BENDIABDALLAH M.H
hakim.bendiabdallah@gmail.com
hakim.bendiabdallah@univ-temouchent.edu.dz
University of Ain Temouchent – Department of Computer Science
Module: Natural Language Processing

Dr. BENDIABDALLAH M.H

Overview

In this practical session, you will build a small text-classification pipeline for SMS spam detection. The goal is to guide you toward the same workflow and outcomes as the reference notebook, while still leaving enough room for you to think, test ideas, and justify your choices.

You are expected to work carefully, explain your observations in clear English, and complete each coding step in your notebook without copying a full ready-made solution.

Libraries to Install (if needed)

If you are using the university lab, Google Colab, or another preconfigured environment, these libraries may already be available.

If you are working on your own machine, install the following Python packages before starting:

- `pandas`
- `matplotlib`
- `seaborn`
- `scikit-learn`
- `notebook` or `jupyter` if you want to run the notebook locally

Example installation command:

- `pip install pandas matplotlib seaborn scikit-learn notebook`

Learning Outcomes

By the end of this TP, you should be able to:

- load and inspect a real-world text dataset,
- prepare textual data for machine learning,
- represent text using `CountVectorizer` and `TfidfVectorizer`,
- train a baseline `MultinomialNB` classifier,
- compare models with suitable evaluation metrics,
- interpret results using charts and a confusion matrix.

General Instructions

- Read each task carefully before writing code.
- Complete the notebook progressively: do not jump to the last task before validating the previous one.
- Use **relative paths only**. If needed, the dataset is available as `spam.csv`.
- When a result looks surprising, print intermediate outputs and explain what you observe.
- You are encouraged to make reasonable choices, but you must be able to justify them.
- Do **not** hard-code final metric values in your plots or conclusions; compute them from your models.

Recommended Files

- Dataset: `.csv`

Recommended Libraries and Useful Commands

You are free to organise your notebook in your own way, but the following libraries and commands are strongly recommended because they match the expected workflow of this TP.

Recommended imports:

- `import pandas as pd`
- `import matplotlib.pyplot as plt`

Dr. BENDIABDALLAH M.H

- `import seaborn as sns`
- `from sklearn.model_selection import train_test_split`
- `from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer`
- `from sklearn.naive_bayes import MultinomialNB`
- `from sklearn.metrics import accuracy_score, f1_score, confusion_matrix`

Typical commands you may need:

- load data with `pd.read_csv(...)`
- inspect data with `df.head()`, `df.columns`, `df.info()`, `df['labels'].value_counts()`
- remove columns with `df.drop(columns=[...])`
- rename columns with `df.columns = [...]`
- plot counts with `sns.countplot(...)` or `value_counts().plot(...)`
- split data with `train_test_split(...)`
- vectorise text with `fit_transform(...)` and `transform(...)`
- train a classifier with `model.fit(...)` and predict with `model.predict(...)`
- visualise the confusion matrix with `sns.heatmap(..., annot=True, fmt='g')`

Tasks

Task 1 – Environment Setup

Import the libraries that you need for data handling, plotting, vectorisation, model training, and evaluation.

Hint: Start with the imports listed in the "Recommended Libraries and Useful Commands" section, then add extra ones only if you really need them.

Task 2 – Load the Dataset

Load the CSV file into a dataframe and display a few rows.

- Make sure the file path is relative.
- Handle possible encoding issues.
- Print the number of rows and columns.

Hint: Use `pd.read_csv(...)`. If the default encoding fails, try `encoding='ISO-8859-1'`.

Dr. BENDIABDALLAH M.H

Task 3 – Data Cleaning

Prepare the dataset for modelling.

- Inspect the column names.
- Remove columns that are empty or useless for prediction.
- Rename the two important columns with simple, meaningful names.

Hint: Useful commands: `df.columns`, `df.drop(columns=[...], errors='ignore')`, and then `df.columns = ['labels', 'text']`.

Task 4 – Visualise Class Distribution

Create a chart showing how many messages belong to each class.

- Use a readable title and axis labels.
- Add one or two sentences commenting on the balance between classes.

Hint: A simple option is `sns.countplot(...)`. You can also start from `value_counts()` and build your own bar chart.

Task 5 – Binary Label Encoding

Convert the textual labels into numerical values suitable for classification.

Hint: A dictionary mapping such as `{'ham': 0, 'spam': 1}` with `map(...)` is enough. Store the final target in a variable such as `y`.

Task 6 – Train/Test Split

Split the message texts and target labels into training and testing subsets.

- Use a test size of **33%**.
- Use **`random_state=15`** for reproducibility.
- Briefly justify any extra option you add.

Hint: The expected tool is `train_test_split(df['text'], y, ...)`. Ask yourself whether `stratify=y` would be useful here.

Dr. BENDIABDALLAH M.H

Task 7 – Vectorisation: BoW and TF-IDF

Build two textual representations from the training set:

1. a Bag-of-Words representation,
2. a TF-IDF representation.

Then transform both the training set and the test set.

Your notebook should clearly show:

- which vectoriser is used,
- the shape of each resulting matrix,
- that the fitting step is done on the correct subset.

Hint: Use one `CountVectorizer(...)` and one `TfidfVectorizer(...)`. The key methods are `fit_transform(X_train)` and `transform(X_test)`.

Task 8 – Baseline Model: Multinomial Na"ive Bayes

Train one `MultinomialNB` model for each representation.

For both models:

- produce predictions on the test set,
- compute **Accuracy** and **F1-score**,
- present the results in a clean table or structured printout.

Hint: A typical sequence is: create the model, fit it on the training vectors, predict on the test vectors, then compute `accuracy_score(...)` and `f1_score(...)`.

Task 9 – Compare the Two Models

Create **two separate bar charts**:

- one for the accuracies,
- one for the F1-scores.

Use these plots to identify the stronger representation.

Write a short interpretation:

Dr. BENDIABDALLAH M.H

- Which model performs better overall?
- Is the difference large or small?
- Which metric is more informative in this context, and why?

Hint: Store your metrics first, for example in variables, lists, or a small dataframe, then plot them with `plt.bar(...)` or `sns.barplot(...)`.

Task 10 – Confusion Matrix and Error Analysis

Select the best model from Task 9 and plot its confusion matrix as a heat map.

Then answer the following briefly:

- Are false positives or false negatives more frequent?
- In an SMS spam filter, which error type is more problematic for the user?
- What simple improvement could you test next?

Hint: Use `confusion_matrix(y_test, y_pred)` and display it with `sns.heatmap(...)`. Base your choice of the "best" model on the metrics you actually reported, not on a guess.

Final Advice

This TP is guided on purpose, but it is not a copy-paste exercise. Use the hints to move in the right direction, verify each result, and explain what your code is doing. A correct answer with clear reasoning is always better than a rushed notebook full of unexplained commands.

Dr. BENDIABDALLAH M.H

Good luck and happy coding!